AGENTE INTELIGENTE PARA DIAMOND RUSH

Documentación técnica

Cómo funciona el agente por dentro: visión, modelo del juego, búsqueda A* y ejecución

Proyecto de Sistemas Inteligentes
Octubre de 2026

Contenido

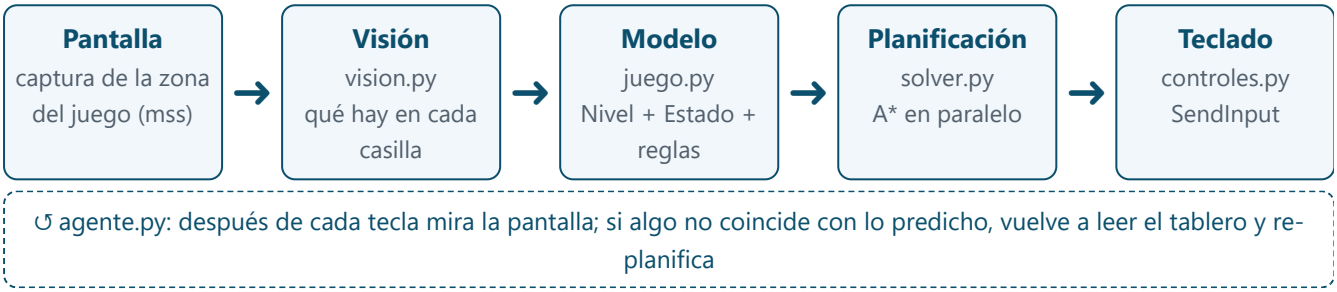
1. Visión general 3
2. Percepción: de píxeles a casillas (vision.py) 4
3. Modelo del juego (juego.py) 5
4. Planificación (solver.py) 6
5. Ejecución y ciclo cerrado (agente.py, controles.py) 8
6. Lo que se aprendió del walkthrough 9
7. Resultados 10
8. Limitaciones y mejoras posibles 11
9. Pruebas 11

Documento técnico. Explica la arquitectura, los algoritmos y las decisiones de diseño, con referencias al código.

1. Visión general

El agente es un **agente basado en objetivos con planificación**. En cada nivel:

1. **Percibe**: captura la pantalla y clasifica cada casilla del tablero.
2. **Modela**: arma un modelo del nivel (lo fijo) y del estado actual (lo que cambia).
3. **Planifica**: busca con **A*** una secuencia de movimientos que recoge todos los diamantes y llega a la salida.
4. **Actúa y verifica**: presiona cada tecla, **espera a ver** el resultado en pantalla y, si no coincide con lo que predijo el modelo, vuelve a leer el tablero y **re-planifica**.



PEAS

Elemento	Descripción
Medida de desempeño	Completar el nivel en el menor tiempo desde la primera tecla (menos movimientos, sin errores)
Entorno	Tablero discreto de 8 × 12 casillas, determinista, totalmente observable salvo excepciones (qué botón abre qué puerta, objetos tapados por rocas)
Actuadores	Flechas del teclado y clic (SendInput en Windows, XTest en Linux)
Sensores	Captura de la zona del juego en pantalla (`mss`)

Módulos

Módulo	Responsabilidad
`vision.py`	Captura, corte en casillas, "huella" de cada casilla y clasificación
`juego.py`	`Nivel` (parte fija + reglas), `Estado` (parte variable), heurísticas, construcción desde la lectura
`solver.py`	A* y ejecución de varias estrategias en paralelo
`agente.py`	Ciclo percibir → planificar → actuar → verificar; re-planificación, reinicios, detección del método de teclas
`controles.py`	Elige la implementación según el sistema operativo
`controles_windows.py`	Teclado y mouse simulados con SendInput, teclas globales (F8/F9), ventana de la consola
`controles_linux.py`	Lo mismo en Linux, con la extensión XTest de X11 (`python-xlib`)

Módulo	Responsabilidad
`herramientas.py`	Calibración, etiquetado, galería de ejemplos
`objetos.py`	Objetos definidos por el usuario ("se comporta como...")

2. Percepción: de píxeles a casillas (`vision.py`)

2.1 Captura y grilla

- `mss` captura solo el rectángulo del juego (`roi` en `config.json`, fijado al calibrar).
- La zona se divide en una **grilla de 10 × 15** casillas. Los bordes de cada casilla se calculan con decimales y se redondean por casilla, para que el error no se acumule.
- Solo se analiza el **tablero efectivo de 8 × 12** (`area = [2, 1, 12, 8]`). Las dos filas de arriba (título "Level N", botones de ajustes y reinicio) y el marco de pared se ignoran y cuentan como pared. La fila de arriba del tablero casi siempre es pared, pero en los niveles 8 y 20 tiene diamantes y rocas medio tapados por el botón WALKTHROUGH; por eso el tablero es de 12 filas y no de 11.

2.2 Huella de una casilla

Para comparar casillas se usa una representación compacta, la "huella" (`huella()`):

1. Se recorta un **margen del 12 %** en cada borde, para tolerar una grilla corrida 1–2 píxeles y no mezclar la casilla vecina.
2. Se reduce a **16 × 16 píxeles** con `INTER_AREA` (promedia bloques, como un desenfoque).
3. Se aplana a un vector de 16 · 16 · 3 = **768 números**.

Como siempre se reduce a 16 × 16, la huella **no depende del tamaño en pantalla**: los ejemplos sacados del video (casillas de 36 px) sirven para una pantalla donde las casillas miden 48 px.

2.3 Clasificación: vecino más cercano

En `plantillas/<objeto>/*.png` hay ejemplos de cada objeto (~700). Cada casilla toma la etiqueta del **ejemplo más parecido**, medido con la distancia RMS entre huellas:

$$d(a, b) = \sqrt{\left(\|a - b\|^2 / 768 \right)}$$

Las 96 casillas × ~700 ejemplos se comparan de una vez con álgebra matricial, usando `‖a − b‖² = ‖a‖² + ‖b‖² − 2·a·b`: un solo producto de matrices. Una lectura completa tarda **~8 ms**.

- Si la distancia al mejor ejemplo supera `umbral_desconocido` (45), la casilla queda como **desconocida** y el modelo la trata como pared, para no planificar sobre algo dudoso.
- **El jugador** se busca aparte: es la casilla con menor distancia a algún ejemplo de jugador. Así se encuentra aunque esté parado sobre otra cosa. Si incluso esa casilla está lejos (más de 2 × 45), se considera que no hay jugador en pantalla.
- **Objetos del usuario** (`objetos.json`): se etiquetan con su propio nombre, pero para el juego se traducen al objeto base al que "se parecen" (por ejemplo, `pared_de_lava` → pared).

Por qué no una red neuronal: los sprites del juego son fijos. El vecino más cercano no necesita entrenamiento (agregar un ejemplo es guardar una imagen), es explicable y es muy rápido. En una prueba de "dejar uno

afuera" sobre un nivel real (cada casilla se clasifica usando las demás como ejemplos) acierta 85 de 88 casillas; los errores son objetos que aparecen una sola vez.

2.4 Calibración

`herramientas.calibrar()`: el usuario marca el juego en una captura y luego ajusta la grilla. El **auto-ajuste** (`f`) es una búsqueda local: prueba corrimientos de ±4 px y cambios de tamaño de ±6 px y se queda con la grilla donde las casillas se parecen más a los ejemplos (menor distancia promedio). Repite hasta 3 vueltas mientras mejore.

2.5 Lectura estable

Durante las animaciones el tablero cambia entre cuadros. `leer_estable()` captura hasta que **dos lecturas seguidas coinciden**, para no planificar sobre una imagen a medio movimiento.

3. Modelo del juego (`juego.py`)

3.1 Nivel y Estado

El nivel se separa en dos partes:

- **`Nivel`, lo fijo**: tipo de cada casilla (`PARED`, `SUELO`, `AGUJERO`, `BOTON`, `PUERTA_METALICA`, `SALIDA`, `PUAS`), vecinos de cada casilla y distancias precalculadas.
- **`Estado`, lo que cambia**: una `namedtuple` que se puede usar como clave de diccionario.

```
Estado(jugador,          # casilla del jugador (fila * 8 + columna)
       rocas,            # frozenset de casillas con roca
       diamantes,        # diamantes que faltan
       llaves,           # llaves en el piso
       puertas_llave,    # puertas de llave cerradas
       tiene_llave,      # bool
       puas_pisadas,     # púas ya pisadas (bloquean)
       agujeros_llenos)  # agujeros tapados con roca
```

Cada movimiento cambia pocos campos y los demás se comparten (`_replace`). Por eso guardar millones de estados en el A* cuesta poca memoria.

3.2 Reglas (`Nivel.mover`)

`mover(estado, dirección)` devuelve `(nuevo_estado, ganó)`, o `None` si el movimiento no es posible. Las reglas, en el orden en que se revisan:

Destino	Regla
fuera del tablero, pared, púas ya pisadas, agujero sin tapar	no se puede
puerta metálica	solo si está abierta (ver 3.3)
salida	se gana si no quedan diamantes; si quedan, está cerrada. **Estatua** del nivel 20: se gana sin juntar diamantes

Destino	Regla
puerta de llave	necesita llave; se abre, la llave se gasta y (regla `entra_al_abrir_puerta`) el jugador entra en el mismo paso
roca	se empuja si la casilla siguiente lo permite: suelo, botón, llave, diamante (`roca_sobre_diamante`), púas sin pisar (`roca_sobre_puas`) o agujero. Si cae en un agujero lo tapa y desaparece. No puede ir a paredes, salidas, puertas, otras rocas ni púas pisadas
diamante / llave	se recoge (solo una llave a la vez)
púas sin pisar	se pueden pisar **una vez**: al salir quedan bloqueadas

Varias reglas se **confirmaron con el walkthrough** (sección 6). Por ejemplo, que las rocas pasan por encima de diamantes y de púas. Las reglas dudosas se pueden cambiar en `config.json`.

3.3 Botones y puertas metálicas: modelo optimista que aprende

En pantalla **no se ve qué botón abre qué puerta**, y tampoco sigue una regla de distancia: en el nivel 14, un botón abre una puerta lejana. Por eso:

- **Modelo optimista**: cada puerta tiene un conjunto de **botones candidatos**, al principio todos. Una puerta se considera abierta si **alguno** de sus candidatos tiene una roca encima. Así el modelo nunca descarta una solución real: el emparejamiento verdadero es un caso particular.
- **Aprendizaje** (`Nivel.observar_puertas`): después de cada evento, el agente mira qué puertas se ven abiertas:
  - puerta **abierta** → su botón está entre los que tienen roca: `candidatos n= pulsados`
  - puerta **cerrada** → su botón no está entre ellos: `candidatos -= pulsados`

  No se deduce nada mientras el jugador pisa un botón, porque eso también abre la puerta un momento.
- Si los candidatos cambian, el agente **re-planifica** con el conocimiento nuevo. Si una suposición optimista lo dejó sin salida, reinicia el nivel conservando lo aprendido **en ese intento**. Al empezar otro nivel, la memoria se borra.

3.4 De la lectura al modelo

- `construir_nivel()` traduce la grilla de etiquetas a tipos de casilla. Lo desconocido se trata como pared.
- `construir_estado()` arma el Estado. Cuando re-planifica a mitad del nivel, usa el estado anterior para lo que la cámara no ve: una llave o un diamante **tapados por una roca**, o la púa que está **debajo del jugador**.

4. Planificación (`solver.py`)

4.1 El problema de búsqueda

- **Estado**: el `Estado` completo, no solo la posición. Con rocas, púas y llaves el mismo lugar puede ser un estado muy distinto.
- **Acciones**: 4 direcciones. **Costo**: 1 por movimiento.
- **Meta**: entrar a la salida sin diamantes pendientes.

4.2 A*

`_a_estrella()` implementa A* con `f(n) = g(n) + w · h(n)`:

- **Cola de prioridad** (`heapq`) ordenada por `f`. A igual `f` se prefiere el nodo **más profundo** (mayor `g`), que suele llegar antes a la meta.
- `mejor_g` guarda el mejor costo conocido de cada estado. Las entradas viejas de la cola se descartan al sacarlas ("borrado perezoso").
- La meta se reconoce **al sacarla** de la cola, no al generarla. Con w = 1 y heurística consistente, eso garantiza el camino más corto.
- El camino se reconstruye siguiendo `padre`.

4.3 Heurística admisible

Se precalculan con BFS las distancias entre todas las casillas en un **mapa optimista**, donde todo lo que no es pared se considera transitable (rocas, puertas, agujeros, púas). Ese mapa nunca es más difícil que el real, así que esas distancias **nunca sobreestiman**.

$$h(n) = \text{máx}(\ d(p, S)\ ,\ \text{máx sobre cada diamante } D \text{ pendiente de } [\ d(p, D) + d(D, S)\ ]\ )$$

con p el jugador, S la salida más cercana y D cada diamante pendiente.

- **Admisible**: para terminar hay que pasar por cada diamante pendiente y después llegar a la salida. Cualquier camino real cuesta al menos d(p, D) + d(D, S) para **cada** D, así que también al menos su máximo.
- **Consistente**: un paso cambia cada distancia en a lo más 1. Al recoger un diamante, el término d(p, S) evita que h baje más de 1. Por eso con w = 1 el A* es óptimo.

4.4 Heurística de recorrido

En niveles grandes la admisible guía poco: solo "ve" el diamante más lejano. La heurística de recorrido (`heuristica_tour`) estima el largo del tour **"siempre al diamante más cercano, y al final a la salida"**. No es admisible (puede sobreestimar), pero está mucho mejor informada y encuentra soluciones mucho más rápido, a veces con algunos pasos extra.

4.5 A* ponderado

Con w > 1 el A* confía más en la heurística: explora menos y encuentra antes una solución, que puede ser un poco más larga (con heurística admisible, a lo más w veces la óptima).

4.6 Poda por púas con máscaras de bits

Pisar una púa puede dejar encerrado un diamante o la salida, y desde ahí el nivel queda sin solución. `todo_alcanzable()` revisa, **cada vez que se pisa una púa**, si todavía se llega a todos los diamantes y a la salida, contando las púas pisadas como paredes. Si no, el estado se descarta.

Se calcula con **máscaras de bits**: el tablero es un entero de 96 bits, uno por casilla. La zona alcanzable crece "inundando" a las 4 vecinas con desplazamientos de bits:

```
nueva = (zona | (zona << 1) & sin_col0 | (zona >> 1) & sin_col_ultima
              | (zona << 8) | (zona >> 8)) & libre
```

hasta que deja de crecer. Las máscaras `sin_col0` y `sin_col_ultima` evitan que un desplazamiento pase de una fila a la siguiente. Antes, esta revisión se hacía casilla por casilla y era cerca de la cuarta parte del tiempo del A* en el nivel más pesado. Con bits, ese costo casi desaparece.

4.7 Varias estrategias en paralelo

En los 20 niveles reales, ninguna combinación (heurística, peso) fue la mejor en todos. Por eso `resolver()` corre **varias a la vez**, una por núcleo (núcleos − 1), con `multiprocessing`:

Prioridad	Estrategia	Para qué
1	recorrido, w = 1.5	rápida en casi todos los niveles
2	admisible, w = 1	**plan óptimo** (el más corto) cuando termina a tiempo
3	recorrido, w = 5	muy rápida en niveles grandes (15, 16)
4–6	recorrido w = 1 y 3, admisible w = 5	respaldo

Con pocos núcleos solo corren las primeras. La búsqueda termina cuando:

- termina la **óptima** (no puede haber nada más corto);
- pasan `esperar_mejor_plan` segundos (15) desde el primer plan encontrado;
- se acaba `tiempo_planificacion` (100 s; el torneo da 2 minutos).

Se juega el **plan más corto** encontrado. Si alguna estrategia recorre todo el espacio sin solución, no la hay (con estas reglas) y se detiene.

El tiempo de planificación es "gratis": en el torneo se mide desde la primera tecla. Por eso conviene esperar unos segundos por un plan más corto.

5. Ejecución y ciclo cerrado (`agente.py`, `controles.py`)

5.1 Teclas

`controles.py` usa `SendInput` de Windows, con `ctypes` y sin librerías extra. Hay tres formas de enviar una tecla (`METODOS`): código virtual + código de escaneo (por defecto), solo código de escaneo y solo código virtual.

Si la **primera tecla** no mueve al personaje, `probar_metodos()` prueba las otras formas, con más tiempo de pulsación y, por último, con un clic en el juego para darle foco. La que funciona se guarda en `config.json`.

F8: el agente espera la tecla F8 con `GetAsyncKeyState`, que funciona aunque la ventana activa sea el navegador. Así el juego ya tiene el foco cuando empieza.

Linux (`controles_linux.py`): las mismas funciones con la extensión **XTest** de X11 a través de `python-xlib` (Python puro, no hay que compilar nada). `fake_input` simula teclas y clics; F8/F9 se detectan consultando el mapa de teclas presionadas del servidor X (`query_keymap`) y quedándose con el paso de "suelta" a "presionada"; la terminal se minimiza con los mensajes estándar de los gestores de ventanas (`WM_CHANGE_STATE`, `_NET_ACTIVE_WINDOW`). Necesita una sesión Xorg: en Wayland ningún programa puede capturar la pantalla ni enviar teclas a otras ventanas.

`controles.py` solo elige la implementación según el sistema, así que el resto del agente es el mismo en Windows y en Linux. El planificador en paralelo (`multiprocessing`) y la captura (`mss`) ya funcionan en los dos.

5.2 Sincronización con el juego

No se usan pausas fijas. Después de cada tecla, `esperar_jugador()` lee la pantalla hasta **ver al jugador en la casilla esperada** (cada lectura tarda ~8 ms) y recién ahí manda la siguiente tecla. Así el agente va tan rápido como la animación del juego.

5.3 Verificación y recuperación

Situación	Reacción
El jugador llegó y solo cambió su posición	sigue con la siguiente tecla
Hubo un **evento** (roca, recoger algo, puerta)	lee el tablero estable y compara rocas, diamantes y puertas con lo que predijo el modelo
Se abrió (o no) una puerta metálica distinta de la esperada	aprende el emparejamiento y re-planifica (3.3)
El jugador no se movió y nada cambió	la tecla "no entró": la repite (hasta 2 veces)
Sigue sin moverse	el juego no permite ese movimiento: se marca como **bloqueado** (`nivel.bloqueados`) y se re-planifica
El tablero quedó distinto a lo predicho	re-lee la pantalla y **re-planifica** desde la situación real

La re-planificación a mitad del nivel corre **en un solo proceso**, porque el reloj ya está corriendo y crear procesos cuesta tiempo. Su límite es de 30 s.

5.4 Reinicios

`jugar_con_reintentos()` reinicia el nivel con el botón del juego solo cuando sirve:

- si el agente **se atascó jugando** (sin solución desde la situación real), hasta 2 veces;
- si al empezar **no había solución**, porque el nivel ya venía trabado, una vez.

Si no ve el juego o las teclas no llegan, no reinicia: no haría clic a ciegas.

6. Lo que se aprendió del walkthrough

El walkthrough del juego (14 minutos, 20 niveles) se usó para **conocer las reglas y entrenar la visión**. El agente no guarda planes ni tableros: en el torneo siempre calcula a partir de lo que ve. Las herramientas están en `analisis_video/`.

6.1 Extracción (`extraer_niveles.py`, `construir_niveles.py`)

1. **Cambios de nivel**: se compara cada 5 cuadros una versión reducida del tablero; un cambio casi total es un cambio de nivel. Luego se toma el **primer cuadro quieto** (4 cuadros sin cambios).
2. **Agrupamiento**: las 2.600 casillas de los 20 tableros iniciales se agruparon por parecido (algoritmo "líder": cada casilla va al primer grupo a menos de una distancia 20). Quedaron **266 grupos**, que se etiquetaron a mano mirando una hoja con ejemplos de cada uno (`etiquetas_grupos.py`).
3. **Correcciones**:
   - el grupo "suelo" mezclaba suelo, púas y agujeros, y se separó contando píxeles oscuros (suelo 0–19, púas 45–70, agujero > 190);

- un círculo negro fijo era el **cursor de la grabación**, no un objeto;
- el sombrero del jugador asomado bajo el título se confundía con suelo.

4. **Ejemplos de estados**: del último cuadro de cada nivel se sacaron púas pisadas, agujeros tapados, salida abierta, puertas metálicas abiertas y botones con roca. En total ~700 ejemplos para `plantillas/`. En capturas de otra pantalla (casillas de 48 px en vez de 36) leyeron bien 94 de las 96 casillas; los 2 errores estaban bajo el botón de ajustes y se corrigieron agregando un ejemplo de esa casilla.

6.2 Reglas confirmadas

Hallazgo	Evidencia
Tablero de 8 × 12, no 8 × 11	Niveles 8 y 20 tienen objetos en la fila bajo el título
Las rocas pasan sobre diamantes	Nivel 4: la roca termina más allá de un diamante al que solo se llega por un lado
Las rocas pasan sobre púas	Nivel 14: una roca empujada sobre púas (457,6 s)
El botón no abre la puerta más cercana	Nivel 14: un botón abre una puerta lejana (465,5 s)
El último nivel se gana sin diamantes	Nivel 20 termina con 0/14

`rastrear.py` lee el tablero cuadro a cuadro y anota cada cambio. Así se vio qué botón abre qué puerta.

`movimientos.py` sigue al jugador por su camisa blanca, extrae la secuencia de movimientos y la **reproduce en el modelo**: en los niveles 1 y 4 el modelo acepta todos los movimientos y gana igual que en el juego, lo que valida las reglas.

7. Resultados

`analisis_video/resolver_niveles.py --procesos 3` (como un PC de 4 núcleos), con los tableros iniciales de los 20 niveles:

Nivel	Diamantes	Rocas	Llaves	Púas	Plan	Tiempo
1	12	0	0	0	22	0,2 s
2	14	0	0	6	34	0,2 s
3	11	0	4	4	63	0,3 s
4	6	5	0	0	47	0,2 s
5	8	2	1	4	53	0,3 s
6	10	2	2	3	99	36 s
7	8	19	1	0	42	0,2 s
8	11	4	1	6	45	0,2 s
9	12	3	0	0	49	0,2 s
10	10	6	0	6	58	10 s
11	11	4	2	5	84	22 s
12	9	3	1	7	60	0,3 s

Nivel	Diamantes	Rocas	Llaves	Púas	Plan	Tiempo
13	11	1	2	9	62	0,6 s
14	9	3	1	0	53	0,2 s
15	7	5	1	5	65	11 s
16	5	3	0	0	98	10 s
17	8	3	2	5	72	10 s
18	14	3	0	3	59	0,3 s
19	7	2	1	3	64	0,4 s
20	—	21	0	0	sin solución	> 100 s

Los niveles de ~10 s ya tenían un plan antes: ese tiempo es la espera por uno más corto (`esperar_mejor_plan`).

8. Limitaciones y mejoras posibles

- **Nivel 20**: un puzzle de 21 rocas y varios agujeros en cadena. Se intentó una búsqueda sobre empujes (estilo Sokoban) y no lo resolvió en el tiempo. Ayudaría una heurística que estime cuántas rocas hay que llevar a cada agujero, o detectar rocas "muertas" (trabadas en una esquina).
- **Regla `entra_al_abrir_puerta`**: no se pudo confirmar con el video. Si el juego hace otra cosa, el agente se recupera re-planificando, pero pierde algo de tiempo.
- **Estados tapados**: lo que está debajo de una roca no se ve. Se mantiene con el estado anterior del modelo, pero si la lectura inicial falla en esa casilla, el error se arrastra.
- **Puertas metálicas**: el modelo optimista puede llevar a un reinicio la primera vez que se juega un nivel con varias puertas.

9. Pruebas

Prueba	Qué comprueba
`pruebas.py`	Reglas y A* con niveles dibujados en texto, incluidos niveles imposibles que deben dar "sin solución"
`pruebas_simulacion.py`	El agente completo (visión + plan + teclas + verificación) contra un juego simulado que se "come" teclas, usa otra regla para las puertas o esconde el emparejamiento de botones
`analisis_video/resolver_niveles.py`	El planificador sobre los 20 niveles reales; cada plan se verifica en el modelo
`analisis_video/movimientos.py N`	Reproduce en el modelo los movimientos del walkthrough en el nivel N

En Linux (Ubuntu 24.04) pasan las mismas pruebas, y además se probó el agente de punta a punta en un servidor X: un juego simulado en una ventana X11, con captura de pantalla real (`mss`) y teclas reales (XTest). El agente leyó el tablero sin errores, planificó y ganó enviando 13 teclas.